WinDeply

Windows OS Deployment Automation Platform

Full Project Report

Version 1.0 ·
Developer: pavan1832 · GitHub: github.com/pavan1832/windeply

Lokpavan P

1. Executive Summary

WinDeply is a production-quality, full-stack enterprise platform built to automate, simulate, and monitor Windows OS deployment workflows. The system provides a modern DevOps-style dashboard allowing IT operations teams to manage deployment jobs, define reusable configuration templates, register endpoints, and maintain a complete audit trail.

The platform mirrors real-world enterprise deployment patterns: multi-step workflow execution, PowerShell automation, role-based operation, and cloud-native deployment using Docker, Vercel, and Render.

Key outcomes:

* 5-page frontend dashboard built with Next.js 14 and TypeScript
* RESTful backend API with 20+ endpoints built with Node.js, Express and TypeScript
* PostgreSQL database with 5 normalized tables, indexes and seed data
* 3 PowerShell automation scripts for Windows configuration workflows
* Docker Compose setup for one-command local development
* GitHub Actions CI/CD pipeline: lint, build, Docker push, deploy
* Live cloud deployment: Vercel (frontend), Render (backend), Neon (database)
* 6 real build/deployment bugs identified and resolved end-to-end

2. Project Overview

2.1 Problem Statement

Enterprise IT teams deploying Windows machines at scale face significant challenges: manual configuration is error-prone and inconsistent. There is no unified view of deployment status, no reusable templates for machine types, and no structured audit trail of what was installed or configured on each endpoint.

2.2 Solution

WinDeply solves this by providing a centralized automation platform that orchestrates deployment workflows, executes PowerShell scripts, tracks step-by-step progress, and logs every action. Operations teams get a single pane of glass for the entire Windows deployment lifecycle.

2.3 Scope

Area	Scope
Frontend	5 pages: Dashboard, Jobs, Logs, Templates, Machines
Backend	**REST API with 20+ endpoints across 5 resource types**
Database	PostgreSQL with 5 tables, indexes, and seed data
Automation	**3 PowerShell scripts: packages, security, health check**
DevOps	Docker Compose, GitHub Actions CI/CD, Vercel + Render + Neon

Lokpavan P

3. System Architecture

3.1 Architecture Diagram

```
+-------------------------------------------------+
|    FRONTEND DASHBOARD  (Next.js / Vercel)       |
|  Dashboard - Jobs - Logs - Templates - Machines |
+-------------------+-----------------------------+
                    | HTTP REST / JSON
+-------------------v-----------------------------+
|    REST API  (Node.js + Express / Render)       |
|  /api/v1/deployments - /logs - /templates       |
+-------------------+-----------------------------+
                    |
+-------------------v-----------------------------+
|    AUTOMATION CONTROLLER                        |
|  Deployment Engine - Step Executor - Log Writer |
+-------------------+-----------------------------+
                    |
+-------------------v-----------------------------+
|    POWERSHELL SCRIPTS                           |
|  install_packages - configure_security - health |
+-------------------+-----------------------------+
                    |
+-------------------v-----------------------------+
|    PostgreSQL DATABASE  (Neon)                  |
|  deployments - logs - templates - machines      |
+-------------------------------------------------+
```

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	Next.js 14, React 18	App router, SSR, client components
	TypeScript	Type safety across all components
	Tailwind CSS	Terminal-aesthetic dark UI styling
	Redux Toolkit	Global state with async thunks
	Recharts	Area charts and pie charts
Backend	Node.js + Express	**REST API server**
	TypeScript	**Strict typing compiled to JS for production**
	Helmet, CORS, Morgan	**Security headers, cross-origin, request logging**
Database	PostgreSQL 16	Primary relational data store
	Neon (cloud)	Serverless Postgres with connection pooling + SSL

Automation	PowerShell 7	**Windows configuration scripts**
DevOps	Docker + Compose	Containerised local development
	GitHub Actions	CI/CD: lint, build, Docker push, deploy
Hosting	Vercel	**Frontend with edge CDN**
	Render	**Backend API with auto-deploy on push**

4. Database Design

4.1 Tables

machines — registered Windows endpoints

Column	Type	Notes
id	UUID PK	Auto-generated via gen_random_uuid()
name	VARCHAR(255)	**Unique machine hostname e.g. DESKTOP-ALPHA**
status	VARCHAR(20)	online | offline | deploying | unknown
ip_address	VARCHAR(45)	**IPv4 or IPv6**
os_version	VARCHAR(100)	Windows version string
department / assigned_to	VARCHAR(100)	**Organisational metadata**

deployments — job lifecycle tracking

Column	Type	Notes
machine_id	UUID FK	References machines.id
template_id	UUID FK	**References deployment_templates.id**
status	VARCHAR(20)	pending | running | success | failed | cancelled
current_step / total_steps	INTEGER	**Tracks progress through 5-step workflow**
configuration_profile	VARCHAR(100)	standard | developer | hardened | minimal
error_message	TEXT	**Populated on failure**

deployment_logs — full audit trail

Column	Type	Notes

Jkpavan P

deployment_id	UUID FK	Parent deployment, CASCADE delete
level	VARCHAR(10)	**info | warn | error | debug**
message	TEXT	Log message body
step_name / script_name	VARCHAR(255)	**Deployment step and script that produced this log**
output	TEXT	Full stdout from PowerShell script execution

4.2 Indexes

* idx_deployments_status — fast filtering by job status
* idx_deployments_machine_id — fast machine history lookups
* idx_deployment_logs_deployment_id — fast log retrieval per job
* idx_deployment_logs_created_at DESC — fast recent log queries

5. Backend API Reference

5.1 Base URLs

```
Production:  https://windeply-1.onrender.com/api/v1
Local Dev:   http://localhost:4000/api/v1
Health:      GET /health
```

5.2 All Endpoints

Method	Endpoint	Description
GET	/dashboard/stats	Stats + recent logs + machine counts
GET	/deployments	List all deployments, filterable by status
POST	/deployments	**Create a new deployment job**
GET	/deployments/:id	Get single deployment with machine + template name
POST	/deployments/:id/execute	**Start async deployment workflow**
POST	/deployments/:id/cancel	Cancel pending or running job
GET	/logs	**All logs — searchable by level and message**
GET	/deployments/:id/logs	Logs for a specific deployment (ordered ASC)
GET	/templates	**List all deployment templates**
POST	/templates	Create new template
GET	/templates/:id	**Get template by ID**

Lokpavan P

PUT	/templates/:id	Update template
DELETE	/templates/:id	**Delete template**
GET	/machines	List machines with deployment stats
POST	/machines	**Register new machine**
PUT	/machines/:id	Update machine details or status
DELETE	/machines/:id	**Remove machine**
GET	/scripts	List all automation scripts

6. Frontend Pages

Page	Route	Key Features
Dashboard	/	Live stat cards, area chart, pie chart, active deployment progress, log feed, jobs table
Deployment Jobs	/jobs	**Create/execute/cancel jobs, status filter, per-job log modal, 5-step progress bar**
Audit Logs	/logs	Terminal-style viewer, live refresh toggle, level filter, text search, line numbers
Templates	/templates	**Card grid, create with software builder and script checkboxes, detail modal, delete**
Machines	/machines	Fleet table with status icons, register modal, deployment counts, last-seen

6.1 Design System

* Theme: Terminal-style dark UI — navy background #0a0e1a, cyan accent #00d4ff, green #00ff88
* Fonts: JetBrains Mono (code/data), IBM Plex Sans (body), Space Grotesk (headings)
* Shared components: StatusBadge, StatCard, DeploymentProgress, Button, LogLevelBadge, Spinner
* Real-time auto-polling every 3-6 seconds for live deployment monitoring
* Redux Toolkit with async thunks for state management
* @/ path aliases configured in tsconfig.json for clean imports

7. PowerShell Scripts

Script	Category	What It Does
install_packages.ps1	Installation	Installs 10+ packages (VSCode, Git, Node.js, Docker, Chrome, Slack etc.) from a registry. Validates checksums, reports results, exits 0/1.

Lokpavan P

configure_security.ps1	Security	**Firewall rules (5 rules), UAC policies (8 settings), service hardening (8 services), registry hardening (8 settings), optional BitLocker. CIS L1 compliant. WhatIf mode supported.**
system_health_check.ps1	Diagnostic	12 pre-deployment checks: OS, CPU, RAM, disk space, network, DNS, Windows Defender, PowerShell version, admin rights, pending reboots, NTP sync. Pass/fail summary.

8. Deployment Workflow

8.1 5-Step Process

#	Step	Script	Description
1	Initialize Deployment	system_health_check.ps1	Validate system readiness, check prerequisites
2	Apply OS Image Config	—	**Configure OS baseline settings and parameters**
3	Install Required Software	install_packages.ps1	Install all packages from the deployment template
4	Apply Security Policies	configure_security.ps1	**Firewall rules, BitLocker, GPO, registry hardening**
5	Finalize Configuration	generate_report.ps1	Audit report, final checks, mark complete

8.2 Status Lifecycle

```
pending  -->  running  -->  success
               |
               +-->  failed

pending or running  -->  cancelled  (user action)
```

The deployment engine runs asynchronously — the API responds immediately when execute is called, and the 5-step workflow runs in the background. The frontend polls every 4 seconds to show live progress.

9. DevOps & CI/CD

9.1 Docker Compose (Local Dev)

* postgres — PostgreSQL 16 Alpine with health check and named volume
* backend — Node.js API, depends_on postgres healthy

Lokpavan P

* frontend — Next.js standalone, depends_on backend healthy

```
docker compose up --build
# Frontend: http://localhost:3000
# Backend:  http://localhost:4000
```

9.2 GitHub Actions Pipeline

Job	Trigger	Actions
lint	All pushes + PRs	ESLint + TypeScript type check on both frontend and backend
build-backend	After lint	**npm install && tsc, uploads dist/ artifact**
build-frontend	After lint	npm install && next build, uploads .next/ artifact
docker	main only	**Build and push images to GitHub Container Registry (GHCR)**
deploy-backend	main only	Trigger Render deploy hook via HTTP POST
deploy-frontend	main only	**Deploy to Vercel production via vercel-action**

9.3 Cloud Infrastructure

Service	Provider	Plan	Notes
Frontend	Vercel	Hobby (Free)	Edge CDN, auto HTTPS, auto deploy on push
Backend API	Render	Free Web Service	**Auto deploy via deploy hook**
Database	Neon	Free Serverless	DATABASE_URL with SSL, connection pooling
Container Reg.	GitHub GHCR	Free	**Docker images pushed on main branch**

10. Issues & Fixes

The following bugs were encountered during development and deployment and were resolved as part of the project:

#	Issue	Root Cause	Fix
1	Malformed {frontend,...} folder in zip	Shell brace expansion in mkdir -p created a literal folder	Removed phantom folder, re-zipped cleanly

Lokpavan P

		name instead of separate directories	
2	Module not found: ../../lib/api	Pages used relative ../../ imports; Next.js requires @/ alias with baseUrl configured	**Updated all imports to @/lib/api, @/components/ui etc. Added baseUrl to tsconfig.json**
3	Backend: password auth failed for postgres	.env file was not included in the project; backend fell back to defaults with no matching local DB	Added backend/.env and frontend/.env.local files
4	Render build: Cannot find declaration file for express	@types/* were in devDependencies; Render runs npm install without --include=dev	**Moved all @types/* and typescript into dependencies**
5	Cannot find name process / console on Render	tsconfig.json missing types: [node] compiler option causing Node.js globals to be unrecognised	Added types: [node] to backend tsconfig.json compilerOptions
6	DATABASE_URL not supported for Neon	db/index.ts only accepted individual host/port/user env vars, not a Neon connection string	**Updated Pool init to detect DATABASE_URL and use connectionString with SSL rejectUnauthorized: false**

11. Project File Structure

```
windeply/
|-- frontend/
|   |-- src/
|   |   |-- app/
|   |   |   |-- page.tsx              # Dashboard
|   |   |   |-- jobs/page.tsx         # Deployment jobs
|   |   |   |-- logs/page.tsx         # Audit log viewer
|   |   |   |-- templates/page.tsx    # Templates CRUD
|   |   |   |-- machines/page.tsx     # Machine registry
|   |   |   |-- layout.tsx            # Root layout + providers
|   |   |   +-- globals.css           # Terminal theme CSS
|   |   |-- components/
|   |   |   |-- ui/index.tsx          # Shared UI components
|   |   |   |-- layout/Sidebar.tsx    # Navigation sidebar
|   |   |   +-- Providers.tsx         # Redux provider wrapper
|   |   |-- store/
|   |   |   |-- index.ts              # Redux store config
|   |   |   +-- slices/               # dashboardSlice, deploymentsSlice
|   |   +-- lib/api.ts                # Typed API client
|   |-- package.json
|   |-- tsconfig.json                 # baseUrl + @/ alias
|   |-- tailwind.config.js
|   +-- .env.local
|
|-- backend/
|   |-- src/
```

```
|   |   |-- controllers/              # 5 resource controllers
|   |   |-- routes/index.ts           # All API routes
|   |   |-- services/deploymentService.ts  # Async automation engine
|   |   |-- db/index.ts               # Pool + DB init + seed data
|   |   |-- types/index.ts            # Shared TypeScript types
|   |   +-- index.ts                  # Express app entry point
|   |-- package.json                  # @types/* in dependencies
|   |-- tsconfig.json                 # types: [node]
|   +-- .env
|
|-- scripts/powershell/
|   |-- install_packages.ps1
|   |-- configure_security.ps1
|   +-- system_health_check.ps1
|
|-- database/schema.sql
|-- docker-compose.yml
|-- .github/workflows/ci-cd.yml
+-- README.md
```

12. Conclusion

WinDeply is a complete, production-ready full-stack DevOps platform built from scratch. The project covers every layer of modern software engineering: relational database design, REST API development, reactive frontend, real-time monitoring, PowerShell automation, containerisation, and cloud deployment.

The system is fully operational in the cloud with all three tiers running: frontend on Vercel, API on Render, and database on Neon. Every git push to main triggers an automated build and deployment pipeline via GitHub Actions.

Skills demonstrated through this project:

* Full-stack TypeScript development across frontend and backend
* PostgreSQL schema design with relational integrity, indexes, and automated seeding
* Async deployment simulation with real-time database updates and log streaming
* Production-grade configuration: environment variables, SSL, CORS, Helmet security headers
* Cloud-native patterns: DATABASE_URL, connection pooling, Neon SSL, Render auto-deploy
* Debugging real build and deployment failures in CI/CD and cloud environments
* DevOps pipeline design with Docker, GitHub Actions, and multi-service orchestration

-- End of Report --

Lokpavan